

Демонстрация: локальный Next.js-проект → GitHub → Vercel

Пошаговый сценарий преподавателя · занятие №5

Проверка проекта, Git, GitHub, Pull Request и публикация на Vercel

ПРОВЕРИТЬ

СОХРАНИТЬ

ОПУБЛИКОВАТЬ

Важно: настоящий Gemini API key на проекторе не показывать.

Готовый маршрут для живой демонстрации: от локальной проверки до публичной ссылки.

Назначение документа

Пошаговый сценарий преподавателя для занятия №5.

На демонстрации:

1. Проверяем проект перед публикацией.
2. Инициализируем Git.
3. Создаём пустой репозиторий GitHub.
4. Выполняем первый push.
5. Показываем ветку и Pull Request.
6. Импортируем проект в Vercel.
7. Добавляем environment variables.
8. Проверяем публичную ссылку.

Настоящий Gemini API key на проекторе не показывать.

1. Что подготовить до занятия

- авторизованный аккаунт GitHub;
- авторизованный аккаунт Vercel;
- установленный Git;
- установленный Node.js 20+;
- установленный npm;
- готовая Next.js-версия проекта;
- рабочий Gemini API key в `.env.local`;
- резервная копия папки проекта;
- запасной успешно опубликованный deployment;
- окно терминала с крупным шрифтом;
- браузер с открытыми GitHub и Vercel.

Персональные placeholders этого документа:

```
<USERNAME> — имя пользователя GitHub
<REPOSITORY> — название репозитория
<GITHUB_URL> — HTTPS URL созданного репозитория
```

Рекомендуемое название репозитория:

```
study-planner-nextjs-demo
```

2. Открыть папку проекта

PowerShell:

```
Set-Location -LiteralPath "D:\Install\Mangystau Hub\Project 9 days"
```

Показать текущую папку:

```
Get-Location
```

Показать основные файлы:

```
Get-ChildItem -Force
```

Акцент для аудитории:

Перед Git проверяем именно ту папку, которую собираемся публиковать.

3. Проверить инструменты

```
node --version
pnpm --version
git --version
```

Ожидается Node.js 20 или новее.

Проверить настройку Git:

```
git config --global user.name
git config --global user.email
```

Если имя и email отсутствуют, настроить своими данными:

```
git config --global user.name "<ВАШЕ_ИМЯ>"
git config --global user.email "<ВАШ_EMAIL>"
```

Не использовать чужие данные или учебные placeholders в реальной конфигурации.

4. Проверить проект до Git

Установить зависимости:

```
pnpm install
```

Выполнить production-сборку:

```
pnpm build
```

Критерий:

Сборка завершается без ошибки.

При необходимости запустить production-версию:

```
pnpm start
```

Открыть:

```
http://localhost:3000
```

Остановить сервер:

```
Ctrl + C
```

Акцент:

Успешный deployment не должен быть первым местом, где мы обнаруживаем ошибку сборки.

5. Проверить секреты

Проверить наличие безопасных файлов, не выводя значение ключа:

```
Test-Path -LiteralPath ".env.local"
Test-Path -LiteralPath ".env.example"
Test-Path -LiteralPath ".gitignore"
```

Посмотреть `.env.example`:

```
Get-Content -LiteralPath ".env.example"
```

Ожидаемое содержимое:

```
GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.7-flash
```

Посмотреть правила исключения:

```
Get-Content -LiteralPath ".gitignore"
```

Обязательные правила:

```
.env
.env.*
!.env.example
node_modules/
.next/
.vercel/
```

Не выполнять на проекторе:

```
Get-Content .env.local
```

Акцент:

Мы подтверждаем наличие секрета, но не показываем его значение.

6. Убедиться, что Git ещё не создан

```
Test-Path -LiteralPath ".git"
```

Ожидаемый результат до демонстрации:

```
False
```

Команда ниже также должна сообщить, что папка пока не является репозиторием:

```
git status
```

После этого можно начинать демонстрацию Git.

7. Инициализировать локальный репозиторий

```
git init
```

Проверить состояние:

```
git status
```

Показать игнорируемые файлы при необходимости:

```
git status --ignored
```

Убедиться, что в будущий commit не входят:

```
.env.local
node_modules/
.next/
.vercel/
```

8. Создать первый commit

Добавить файлы в staging:

```
git add .
```

Обязательно проверить результат до commit:

```
git status
```

Если среди staged files виден `.env.local`, остановиться и исправить `.gitignore`.

Создать commit:

```
git commit -m "Initial Next.js MVP"
```

Назвать основную ветку `main`:

```
git branch -M main
```

Проверить историю:

```
git log --oneline --decorate -5
```

Акцент:

Commit сохраняет понятную контрольную точку локально. На GitHub проект ещё не отправлен.

9. Создать пустой репозиторий GitHub

В браузере открыть:

```
https://github.com/new
```

Заполнить:

```
Owner:      <USERNAME>
Repository: <REPOSITORY>
Visibility: Public
```

Не включать:

- Add a README file;
- Add `.gitignore`;
- Choose a license.

Нажать:

```
Create repository
```

Почему репозиторий должен быть пустым:

`README` и `.gitignore` уже находятся в локальном проекте. Если создать отдельный начальный commit на GitHub, перед первым push появятся две независимые истории.

После создания скопировать HTTPS URL:

```
https://github.com/<USERNAME>/<REPOSITORY>.git
```

10. Подключить GitHub как remote

Вставить настоящий URL вместо placeholder:

```
git remote add origin https://github.com/<USERNAME>/<REPOSITORY>.git
```

Проверить remote:

```
git remote -v
```

Ожидается:

```
origin <GITHUB_URL> (fetch)
origin <GITHUB_URL> (push)
```

Если `origin` был добавлен ошибочно:

```
git remote set-url origin <GITHUB_URL>
```

11. Выполнить первый push

```
git push -u origin main
```

Флаг `-u` связывает локальную ветку `main` с `origin/main`.

После успешного push:

1. Обновить страницу GitHub.
2. Показать список файлов.
3. Открыть README.
4. Убедиться, что `.env.local` отсутствует.
5. Убедиться, что `node_modules` и `.next` отсутствуют.

Проверить локально:

```
git status
```

Ожидаемый результат:

```
nothing to commit, working tree clean
```

12. Продемонстрировать отдельную ветку

Создать ветку:

```
git switch -c feature/readme
```

Внести небольшое заметное изменение в `README.md`, например добавить строку:

```
## Публичная демонстрация

Проект подготовлен для deployment на Vercel.
```

Проверить изменение:

```
git status
git diff
```

Добавить только README:

```
git add README.md
```

Создать понятный commit:

```
git commit -m "Improve project README"
```

Отправить ветку:

```
git push -u origin feature/readme
```

Акцент:

Изменение находится в отдельной ветке и пока не меняет стабильный `main`.

13. Создать Pull Request

В GitHub:

1. Нажать [Compare & pull request](#).
2. Проверить:
- base: `main`; - compare: `feature/readme`.
3. Заголовок:

```
Improve project README
```
4. В описании написать:

```
Добавлен раздел о публичной демонстрации проекта.
```
5. Открыть вкладку [Files changed](#).
6. Показать фактический diff.
7. Убедиться, что изменён только `README.md`.
8. Нажать [Create pull request](#).
9. Выполнить [Merge pull request](#) → [Confirm merge](#).

Акцент:

Pull Request позволяет проверить фактические изменения до их попадания в `main`.

14. Обновить локальный main после merge

Merge на сайте не обновляет локальную папку автоматически.

Переключиться на `main`:

```
git switch main
```

Получить изменения:

```
git pull
```

Проверить историю:

```
git log --oneline --decorate --graph -8
```

Проверить состояние:

```
git status
```

15. Импортировать GitHub-репозиторий в Vercel

В браузере открыть Vercel Dashboard.

Последовательность:

```
Add New
→ Project
→ Import Git Repository
```

Если GitHub ещё не подключён:

1. Разрешить Vercel доступ к GitHub.
2. Разрешить доступ к выбранному репозиторию.
3. Вернуться к импорту.

Выбрать:

```
<USERNAME>/<REPOSITORY>
```

Нажать:

```
Import
```

16. Проверить настройки Vercel

Перед Deploy проверить:

```
Framework Preset: Next.js
Root Directory:   ./
Build Command:    определяется автоматически
Output Directory: определяется автоматически
Install Command:  определяется по prpm-lock.yaml
```

Не задавать отдельный Express start command.

Если проект находится в корне репозитория, Root Directory оставляется ./.

17. Добавить environment variables

В разделе [Environment Variables](#) добавить:

```
Name:  GEMINI_API_KEY
Value: <ВСТАВИТЬ_КЛЮЧ_НЕ_ПОКАЗЫВАЯ_ЕГО_АУДИТОРИИ>

Name:  GEMINI_MODEL
Value: gemini-3.7-flash
```

Выбрать окружения:

```
Production
Preview
```

Не создавать:

```
NEXT_PUBLIC_GEMINI_API_KEY
```

Акцент:

GEMINI_API_KEY нужен серверному Route Handler. Префикс **NEXT_PUBLIC_** может раскрыть значение браузеру.

Чтобы не показать ключ на проекторе:

- заранее сохранить его в безопасном менеджере;
- временно отключить демонстрацию экрана на момент вставки;
- либо заранее добавить переменную в подготовленный Vercel Project.

18. Выполнить deployment

Нажать:

Deploy

Во время сборки показать:

- установку зависимостей;
- запуск Next.js build;
- статус deployment;
- deployment logs.

Успешный результат:

Status: Ready

Открыть выданный публичный URL.

19. Проверить публичное приложение

Gemini

Ввести:

Тема: Основы JavaScript
Срок: 3 дня

Нажать:

Сформировать план

Проверить:

- отображается загрузка;
- возвращаются ровно три раздела;
- API-ключ не отображается;
- результат читается на странице.

Погода

Последовательно проверить:

Актау
Атырау
Алматы

Нажать:

Показать погоду

Проверить город, дату, минимальную и максимальную температуру, условия и вероятность осадков.

20. Показать автоматический цикл

Объяснить:

```
push в feature-ветку
→ Preview Deployment
→ проверка
→ Pull Request
→ merge в main
→ новый Production Deployment
```

После изменения environment variables требуется новый deployment: предыдущая опубликованная версия не меняется автоматически.

21. Типовые ошибки

Ошибка: `pnpm build` падает локально

Действия:

1. Прочитать первую содержательную ошибку.
2. Исправить её локально.
3. Повторить:

```
pnpm build
```

Не переходить к Vercel, пока локальная сборка не проходит.

Ошибка: `remote origin already exists`

Проверить:

```
git remote -v
```

Исправить URL:

```
git remote set-url origin <GITHUB_URL>
```

Ошибка: `push` отклонён

Частая причина — GitHub-репозиторий был создан с README или другим начальным commit.

Для учебной демонстрации безопаснее удалить пустой удалённый репозиторий и создать его заново без README, `.gitignore` и лицензии. Не выполнять force push без понимания последствий.

Ошибка: Vercel не видит репозиторий

Проверить GitHub permissions приложения Vercel и разрешить доступ к нужному репозиторию.

Ошибка: `framework` не определён

Проверить:

- `package.json` находится в выбранной Root Directory;
- в dependencies есть `next`;
- scripts содержат `build: next build`.

Ошибка: неверная Root Directory

Для данного проекта использовать:

```
./
```

Если Vercel не находит `package.json`, выбрана неправильная папка.

Ошибка: Gemini не работает после deployment

Проверить в Vercel:

- существует `GEMINI_API_KEY`;
- существует `GEMINI_MODEL`;
- значения назначены Production и Preview;
- после изменения переменных создан новый deployment.

Настоящее значение ключа не выводить в logs.

Ошибка: Open-Meteo не отвечает

Открыть deployment logs, проверить статус внешнего запроса и повторить позже. Gemini и погода являются независимыми сценариями.

Ошибка: deployment имеет статус Failed

Открыть logs и найти первую реальную ошибку. Статус `Failed` сообщает результат, но не причину.

22. Финальная проверка

GitHub:

- репозиторий публичный;
- ветка `main` существует;
- Pull Request был создан и объединён;
- README отображается;
- `.env.local` отсутствует;
- `node_modules` отсутствует;
- `.next` отсутствует.

Vercel:

- deployment имеет статус Ready;
- Production URL открывается;
- Gemini работает;
- погода работает;
- environment variables добавлены;
- ключ не имеет префикса `NEXT_PUBLIC_`.

Локально:

```
git switch main
git pull
git status
```

Ожидается актуальный и чистый `main`.

23. Короткая шпаргалка команд

```
Set-Location -LiteralPath "D:\Install\Mangystau Hub\Project 9 days"

pnpm install
pnpm build

git init
git status
git add .
git status
git commit -m "Initial Next.js MVP"
git branch -M main

git remote add origin https://github.com/<USERNAME>/<REPOSITORY>.git
git remote -v
git push -u origin main

git switch -c feature/readme
# Внести небольшую правку в README.md
git status
git diff
git add README.md
git commit -m "Improve project README"
git push -u origin feature/readme

# После merge Pull Request на GitHub
git switch main
git pull
git status
```

24. Официальные источники

- [GitHub: About Git](#)
- [GitHub: Creating a new repository](#)
- [GitHub: Creating a Pull Request](#)
- [Vercel: Next.js](#)
- [Vercel: GitHub integration](#)
- [Vercel: Environment Variables](#)